06 — SAP Chatbot Architecture (Deep Dive)

Module purpose: the capstone template — combines LLM + SQL (sheet 03) + Vector/RAG (sheets 04-05) + memory into four selectable modes, driven entirely by one config object. Seven files.

File 1 — `chatbot_config.py` (`ChatbotConfig` dataclass)

```
@dataclass
class ChatbotConfig:
    llm_model: str = "gpt-4o-mini"; llm_deployment_id: str = None
    temperature: float = 0.1; max_tokens: int = 1024
    memory_window: int = 10                          # 0 = stateless
    enable_rag: bool = False; embedding_deployment_id: str = None
    vector_table: str = "VECTOR_STORE"; rag_top_k: int = 5; rag_min_score: float = 0.5
    enable_sql: bool = False; sql_tables: list = field(default_factory=list); sql_schema: str
↪   = None
    system_prompt: str = "You are a helpful SAP AI assistant."
    bot_name: str = "SAP AI Assistant"

    def __post_init__(self):                         # fills in gaps from .env if not passed
    ↪  explicitly
        if not self.llm_deployment_id: self.llm_deployment_id =
        ↪  os.getenv("LLM_DEPLOYMENT_ID")
        if not self.embedding_deployment_id: self.embedding_deployment_id =
        ↪  os.getenv("EMBEDDING_DEPLOYMENT_ID")

    def mode(self) -> str:
        parts = []
        if self.enable_rag: parts.append("RAG")
        if self.enable_sql: parts.append("SQL")
        return "+".join(parts) if parts else "GENERAL"
```

Working explanation: `__post_init__` runs automatically right after the dataclass's generated `__init__` — it's the hook used here to fall back to `.env` values only when the caller didn't explicitly pass a deployment ID, letting code paths override config per-call while still having sane defaults. `mode()` derives a display string from the two boolean flags rather than storing mode as a separate field — a single source of truth, impossible to get out of sync.

File 2 — `prompt_builder.py` (dynamic prompt assembly)

```
def build_prompt(config: ChatbotConfig, has_rag: bool, has_sql: bool) -> ChatPromptTemplate:
    system_parts = [config.system_prompt]
    if has_rag: system_parts.append("\n\nYou have access to a knowledge base... use
    ↪  'KNOWLEDGE BASE CONTEXT' below.")
    if has_sql: system_parts.append("\n\nYou also have access to structured data... use
    ↪  'DATABASE RESULTS' below.")
    messages = [("system", "".join(system_parts))]
    if has_rag: messages.append(("system", "KNOWLEDGE BASE CONTEXT:\n{rag_context}"))
    if has_sql: messages.append(("system", "DATABASE RESULTS (SQL:
    ↪  {sql_used}):\n{sql_result}"))
    messages.append(MessagesPlaceholder(variable_name="history"))    # <- LangChain history
↪   injection slot
```

```
    messages.append(("human", "{user_message}"))
    return ChatPromptTemplate.from_messages(messages)
```

Working explanation: the prompt's **shape itself changes** based on which features are active for a given turn — this is different from sheet 05's static prompts. `MessagesPlaceholder` is LangChain's mechanism for splicing a *list* of prior `HumanMessage`/`AIMessage` objects directly into the message sequence (not just a formatted string like `{chat_history}` was in sheet 05) — this keeps proper role structure (user/assistant) intact for models that use it.

File 3 — `rag_retriever.py` / File 4 — `sql_retriever.py` (chatbot-flavoured wrappers)

```
def rag_retrieve(conn, user_message, config: ChatbotConfig, embedder) -> tuple[str, list]:
    chunks = similarity_search(conn, user_message, embedder, config.vector_table,
↪   config.rag_top_k, config.rag_source_filter)
    chunks = [c for c in chunks if c["score"] >= config.rag_min_score] if
↪   config.rag_min_score > 0 else chunks
    if not chunks: return "", []                        # empty string signals "no RAG context
     ↪  this turn"
    context_text = "\n\n".join(f"[{c['rank']}] {c['source_name']} (score:
↪   {c['score']})\n{c['content']}" for c in chunks)
    sources = [{"rank": c["rank"], "source_name": c["source_name"], "score": c["score"]} for
↪   c in chunks]
    return context_text, sources

def sql_retrieve(conn, user_message, config: ChatbotConfig, llm) -> tuple[str, str]:
    tables = [t if isinstance(t, dict) else ({"table": t, "schema": config.sql_schema} if
↪   config.sql_schema else t)
              for t in config.sql_tables]
    schema_ctx = build_schema_context(conn, tables)
    sql_used = generate_sql(user_message, schema_ctx, llm)
    try:
        df = execute_sql(conn, sql_used)
        sql_result = df.to_string(index=False) if not df.empty else "Query returned no rows."
    except Exception as e:
        sql_result = f"SQL execution error: {e}"        # error becomes PART OF the context,
↪   not a crash
    return sql_result, sql_used
```

Working explanation: both functions read settings straight from `config` (vector table, top_k, sql_tables) rather than taking many separate parameters — the config object is the single source of truth for chatbot behavior. `sql_retrieve` deliberately **catches SQL errors and returns them as text** rather than propagating an exception — the chatbot can then explain to the user "the query failed because X" instead of hard-crashing mid-conversation.

File 5 — `chatbot_core.py` (`SAPChatbot` — the orchestrator)

```
class SAPChatbot:
    def __init__(self, config: ChatbotConfig, conn=None):
        if (config.enable_rag or config.enable_sql) and conn is None:
            raise ValueError("HANA connection required for RAG/SQL mode.")   # fail fast, not
             ↪  mid-conversation
        self.config, self.conn, self._history = config, conn, []
```

```
        self._llm = build_llm(config)
        self._embedder = build_embedding_model(config) if config.enable_rag else None

    def chat(self, user_message: str) -> dict:
        rag_context, sql_result, sql_used, sources, mode_parts = "", "", "", [], []
        if self.config.enable_rag and self._embedder:
            rag_context, sources = rag_retrieve(self.conn, user_message, self.config,
↪   self._embedder)
            if rag_context: mode_parts.append("RAG")
        if self.config.enable_sql:
            sql_result, sql_used = sql_retrieve(self.conn, user_message, self.config,
↪   self._llm)
            if sql_result: mode_parts.append("SQL")
        if not mode_parts: mode_parts.append("GENERAL")

        prompt = build_prompt(self.config, has_rag=bool(rag_context),
↪   has_sql=bool(sql_result))   # SHAPE varies per turn
        prompt_vars = {"user_message": user_message, "history": self._get_history_window()}
        if rag_context: prompt_vars["rag_context"] = rag_context
        if sql_result: prompt_vars.update(sql_result=sql_result, sql_used=sql_used)

        answer = (prompt | self._llm | StrOutputParser()).invoke(prompt_vars).strip()
        self._history += [HumanMessage(content=user_message), AIMessage(content=answer)]
        return {"answer": answer, "sources": sources, "sql_used": sql_used,
                "mode": "+".join(mode_parts), "turn": len(self._history) // 2}

    def _get_history_window(self) -> list:
        return [] if self.config.memory_window == 0 else
        ↪  self._history[-(self.config.memory_window * 2):]
    def reset(self): self._history = []
```

Working explanation: `chat()` re-evaluates `has_rag`/`has_sql` **per call**, not once at construction — a chatbot in FULL mode might still get zero RAG hits for a given question (e.g. an off-topic query), and the prompt correctly omits the RAG section for that turn rather than injecting an empty context block. `_get_history_window()` mirrors the sliding-window memory pattern from sheet 05, implemented here as plain list slicing instead of a LangChain memory object.

File 6 — `chatbot_presets.py` (4 factory functions)

```
def create_general_chatbot(system_prompt="...", llm_model="gpt-4o-mini", memory_window=10,
↪   **kw) -> SAPChatbot:
    return SAPChatbot(ChatbotConfig(system_prompt=system_prompt, llm_model=llm_model,
                                    memory_window=memory_window, enable_rag=False,
↪   enable_sql=False, **kw))

def create_rag_chatbot(conn, vector_table="VECTOR_STORE", rag_top_k=5, rag_min_score=0.5,
↪   **kw) -> SAPChatbot:
    return SAPChatbot(ChatbotConfig(enable_rag=True, vector_table=vector_table,
                                    rag_top_k=rag_top_k, rag_min_score=rag_min_score,
↪   enable_sql=False, **kw), conn=conn)

# create_sql_chatbot(conn, sql_tables, ...) and create_full_chatbot(conn, sql_tables, ...)
↪   follow the same shape
```

Working explanation: these are pure convenience wrappers over `ChatbotConfig` + `SAPChatbot` — they exist so a caller who just wants "a standard RAG bot" doesn't need to know every config field; power users can still drop to raw `ChatbotConfig` for full control.

File 7 — `cli_runner.py` (interactive terminal front-end)

```
def run_cli(bot: SAPChatbot):
    while True:
        user_input = input("You: ").strip()
        if user_input.lower() == "/quit": break
        elif user_input.lower() == "/reset": bot.reset(); continue
        elif user_input.lower() == "/history": bot.print_history(); continue
        response = bot.chat(user_input)
        print(f"Bot [{response['mode']}]: {response['answer']}")
        if response["sources"]: print(f"Sources: {[s['source_name'] for s in
        ↪  response['sources']]}")
        if response["sql_used"]: print(f"SQL: {response['sql_used']}")
```

Working explanation: the CLI is a **thin presentation layer** — it never touches `ChatbotConfig` internals or the retrieval functions directly, only `bot.chat()`, `bot.reset()`, `bot.print_history()`. This is why the exact same `SAPChatbot` class also powers a Streamlit `app.py` elsewhere in the project with zero changes to the chatbot logic itself.

Key Points

- One config dataclass (`ChatbotConfig`) drives four distinct behaviors — no branching code scattered across the app, just flag reads.
- `chat()` decides `has_rag`/`has_sql` **fresh every turn** — a FULL-mode bot can still answer GENERAL-style on any given turn if retrieval/SQL find nothing relevant.
- `sql_retrieve` swallows SQL errors into readable text instead of raising — keeps the conversation alive even when a generated query is bad.
- `MessagesPlaceholder` vs. a plain `{chat_history}` string (sheet 05) are two different LangChain history-injection techniques — the placeholder preserves role structure.
- Presentation (CLI, Streamlit) is fully decoupled from chatbot logic — same class, different front-ends.

Interview Q&A

Q: Why does `SAPChatbot.__init__` raise immediately if RAG/SQL is enabled without a connection, instead of failing later when `.chat()` is first called? A: Fail-fast principle — a misconfiguration should surface at construction time (during app startup/testing) rather than mid-conversation in production, where the failure is much more disruptive.

Q: How would FULL mode (RAG+SQL) decide which source actually answered a given question? A: In this template, both sources are always attempted and both are handed to the LLM in one prompt; the LLM's own judgment (guided by the system prompt) decides what to lean on. A more advanced version would add explicit intent classification or a routing graph (e.g. LangGraph) before deciding which retrieval path(s) to invoke.

Q: What's the practical difference between the four `chatbot_presets.py` factory functions and just calling `ChatbotConfig()` directly? A: The presets pre-set the boolean flags (`enable_rag`/`enable_sql`) and provide sensible defaults/docstrings for each common use case — they're a convenience layer, not a different mechanism; anything a preset does, `ChatbotConfig(...)` can also do explicitly.

Q: Why is the CLI runner considered a “thin” layer — what would happen if chatbot logic leaked into it? A: It would tie business logic to one specific UI technology, making it impossible to reuse the same chatbot behavior in a Streamlit app, a UI5 app (sheet 10’s CAP+UI5 pattern), or an API endpoint without duplicating logic — the whole point of `SAPChatbot` as a class is that any front-end can call `.chat()` and get identical behavior.